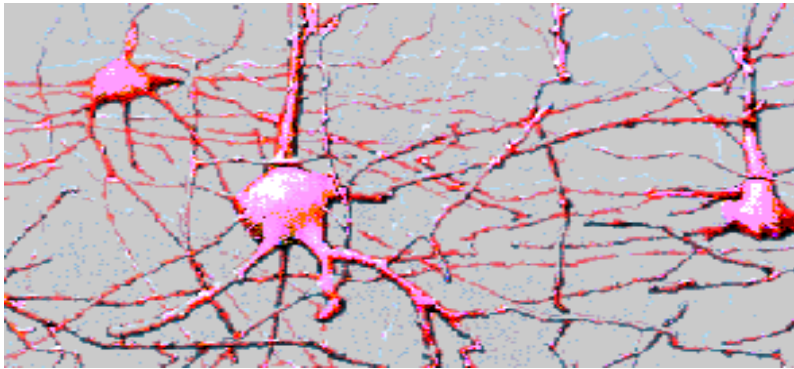
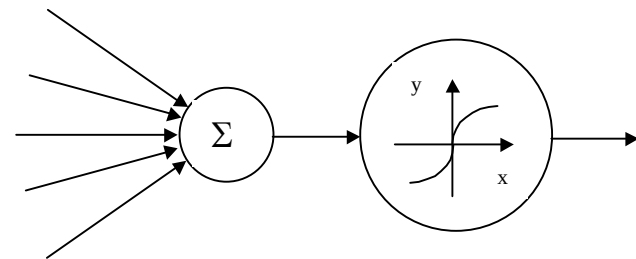
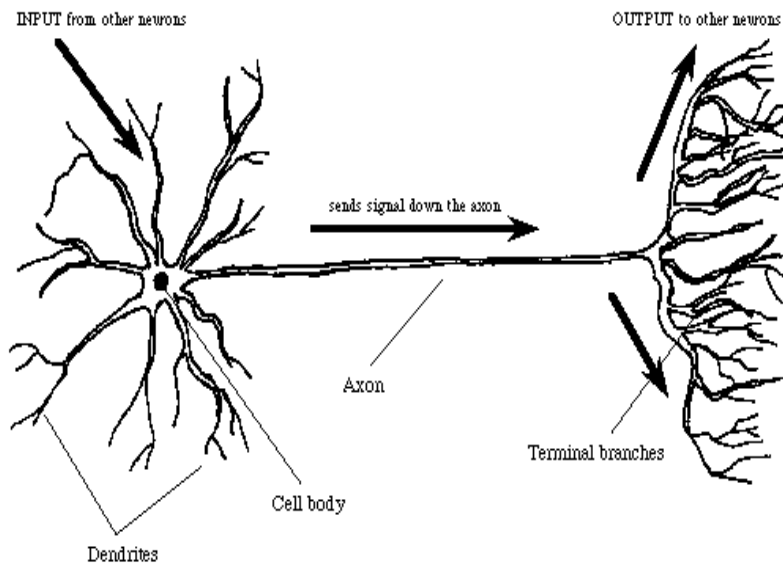
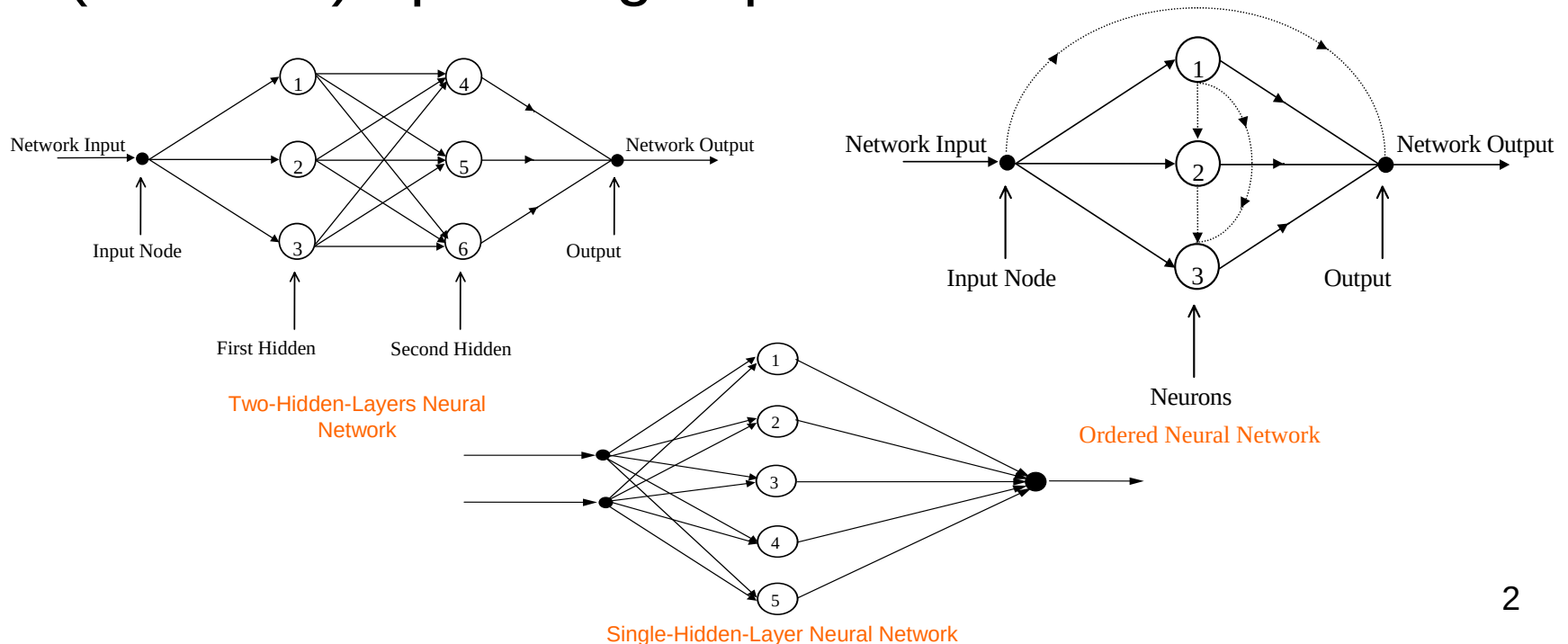


Artificial Neural Networks



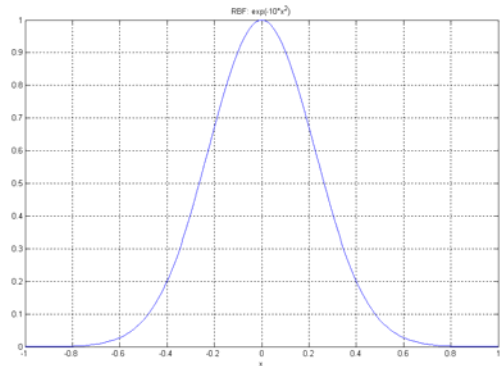
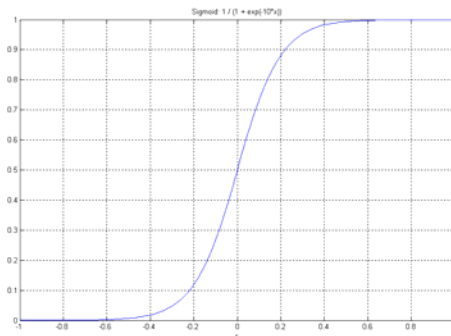
NN Architectures

- Artificial Neural Networks are multi-input-multi-output systems composed of many interconnected nonlinear processing elements (neurons) operating in parallel

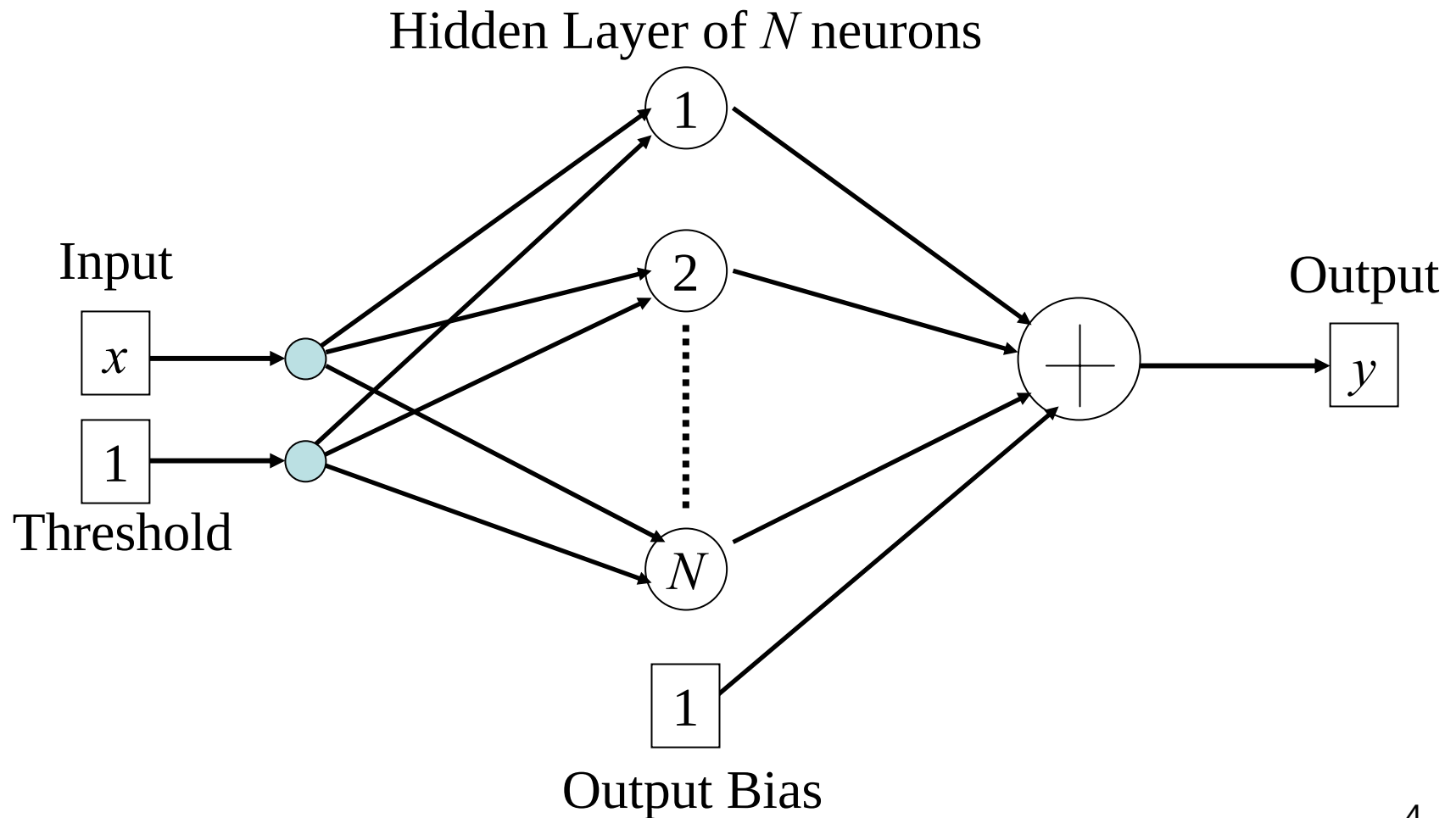


Single Hidden Layer (SHL) Feedforward Neural Networks (FNN)

- Three distinct characteristics
 - model of each neuron includes a nonlinear activation function
- sigmoid $\longrightarrow \sigma(s) = \frac{1}{1 + e^{-s}}$
- radial basis function $\longrightarrow \varphi(x) = e^{-\frac{\|x-r\|^2}{2\sigma^2}}$
- a single layer of N hidden neurons
- feedforward connectivity



SHL FNN Architecture



SHL FNN Function

- Maps n - dimensional input into m - dimensional output: $x \rightarrow NN(x), \quad x \in R^n, \quad NN(x) \in R^m$
- Functional Dependence
 - sigmoidal: $NN(x) = W^T \vec{\sigma}(V^T x + \theta) + b$
 - RBF:

$$NN(x) = W^T \underbrace{\begin{pmatrix} \varphi(\|x - C_1\|) \\ \vdots \\ \varphi(\|x - C_N\|) \end{pmatrix}}_{\Phi(x)} + b = W^T \Phi(x) + b$$

Sigmoidal NN

- Matrix form:
$$NN(x) = W^T \vec{\sigma} \left(V^T \begin{pmatrix} x \\ 1 \end{pmatrix} \right) + c$$

- Vector of hidden layer sigmoids:

$$\vec{\sigma}(V^T x + \theta) = \left(\sigma(v_1^T x + \theta_1) \quad \dots \quad \sigma(\vec{v}_N^T x + \theta_N) \right)^T$$

- Matrix of inner-layer weights:
$$V = (\vec{v}_1 \quad \dots \quad \vec{v}_N) \in R^{n \times N}$$

- Matrix of output-layer weights:
$$W = (\vec{w}_1 \quad \dots \quad \vec{w}_m) \in R^{N \times m}$$

- Vector of output biases $c \in R^m$ and thresholds $\theta \in R^N$

- k^{th} output:

$$NN_k(x) = \vec{w}_k^T \sigma(\vec{v}_k^T x + \theta_k) + c_k = \sum_{j=1}^N w_{jk} \sigma \left(\sum_{i=1}^n v_{ik} x_i + \theta_k \right) + c_k$$

Sigmoidal NN, (continued)

- Universal Approximation Property
 - large class of functions can be approximated by sigmoidal SHL NN-s within any given tolerance, on compacted domains

$$\forall f(x): R^n \rightarrow R^m \quad \forall \varepsilon > 0 \quad \exists N, W, b, V, \theta \quad \forall x \in X \subset R^n$$

$$\left\| f(x) - W^T \vec{\sigma} \left(V^T \begin{pmatrix} x \\ 1 \end{pmatrix} \right) - b \right\| \leq \varepsilon = O\left(\frac{1}{\sqrt{N}}\right)$$

- Introduce: $W \triangleq [W^T \quad b]^T$, $V \triangleq [V^T \quad \theta]^T$, $\vec{\sigma} \triangleq \begin{bmatrix} \vec{\sigma} \\ 1 \end{bmatrix}$, $\mu \triangleq \begin{bmatrix} x \\ 1 \end{bmatrix}$
- Then: $\longrightarrow NN(x) = W^T \vec{\sigma}(V^T \mu)$

Sigmoidal SHL NN: Summary

- A very large class of functions can be approximated using linear combinations of shifted and scaled sigmoids
- NN approximation error decreases as the number N of hidden-layer neurons increases:

$$\|f(x) - NN(x)\| = O\left(N^{-\frac{1}{2}}\right)$$

- Inclusion of biases and thresholds into NN weight matrices simplifies bookkeeping

$$NN(x) = W^T \vec{\sigma}(V^T \mu)$$

- Function approximation using sigmoidal NN means finding connection weights W and V

RBF NN

- Matrix form: $NN(x) = W^T \Phi(x) + b$
- Vector of RBF-s:  $\Phi(x) = \begin{pmatrix} e^{\frac{-\|x-C_1\|^2}{2\sigma_1^2}} & \dots & e^{\frac{-\|x-C_N\|^2}{2\sigma_N^2}} \end{pmatrix}^T$
- Matrix of RBF centers: $C \triangleq [\vec{C}_1 \quad \dots \quad \vec{C}_N] \in R^{n \times N}$
- Vector of RBF widths: $\vec{\sigma} \triangleq (\sigma_1 \quad \dots \quad \sigma_N)^T \in R^N$
- Matrix of output weights: $W = (\vec{w}_1 \quad \dots \quad \vec{w}_m) \in R^{N \times m}$
- Vector of output biases: $b \in R^m$
- k^{th} output: $NN_k(x) = \vec{w}_k^T \Phi(x) + b_k = \sum_{j=1}^N w_{jk} e^{\frac{-\|x-C_j\|^2}{2\sigma_j^2}} + b_k$

RBF NN, (continued)

- Universal Approximation Property
 - large class of functions can be approximated by RBF NN-s within any given tolerance, on compacted domains

$$\forall f(x): R^n \rightarrow R^m \quad \forall \varepsilon > 0 \quad \exists N, W, \vec{C}, \vec{\sigma} \quad \forall x \in X \subset R^n$$

$$\|f(x) - W^T \Phi(x) - b\| \leq \varepsilon = O\left(N^{-\frac{1}{n}}\right)$$

- Introduce: $W \triangleq [W \quad b], \quad \Phi(x) \triangleq \begin{bmatrix} \Phi(x) \\ 1 \end{bmatrix}$

- Then:  $NN(x) = W^T \Phi(x)$

RBF NN: Summary

- A very large class of functions can be approximated using linear combinations of shifted and scaled gaussians
- NN approximation error decreases as the number N of hidden-layer neurons increases:

$$\|f(x) - NN(x)\| = O\left(N^{-\frac{1}{n}}\right)$$

- Inclusion of biases into NN output weight matrix simplifies bookkeeping

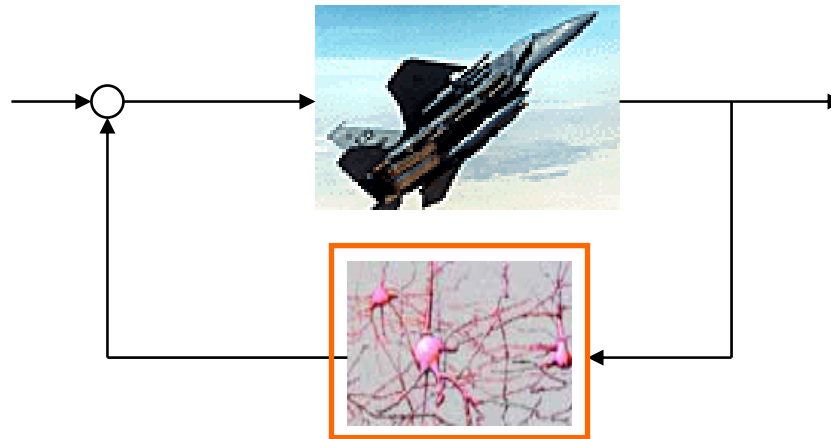
$$NN(x) = W^T \Phi(x)$$

- Function approximation using RBF NN means finding output weights W , centers C , and widths $\vec{\sigma}$

What is Next?

- Use SHL FNN-s in the context of MRAC systems
 - off-line / on-line approximation of uncertain nonlinearities in system dynamics
 - modeling errors, (aerodynamics)
 - damage
 - control failures
- Start with fixed widths RBF NN architectures, (linear in unknown parameters)
- Generalize to using sigmoidal NN-s

Adaptive NeuroControl



n^{th} Order Systems with Matched Uncertainties

- System Dynamics: $\dot{x} = Ax + B\Lambda(u - f(x)), \quad x \in R^n, \quad u \in R^m$
 - $A \in R^{n \times n}$, $\Lambda = \text{diag}(\lambda_1 \dots \lambda_m) \in R^{m \times m}$ are constant unknown matrices
 - $B \in R^{n \times m}$ is known constant matrix
 - $\forall i = 1, \dots, m \quad \text{sgn}(\lambda_i)$ is known
- **Approximation of uncertainty:** $f(x) = \Theta^T \Phi(x) + \varepsilon_f(x)$
 - matrix of constant unknown parameters: $\Theta \in R^{m \times N}$
 - vector of N fixed RBF-s: $\Phi(x) = (\varphi_1(x) \dots \varphi_N(x))^T$
 - function approximation tolerance: $\varepsilon_f(x) \in R^m$

n^{th} Order Systems with Matched Uncertainties, (continued)

- Assumption: Number of RBF-s, true (unknown) output weights Θ and widths/centers are such that RBF NN approximates the nonlinearity within given tolerance:

$$\|\varepsilon_f(x)\| = \|f(x) - \Theta^T \Phi(x)\| \leq \varepsilon, \quad \forall x \in X \subset R^n$$

- RBF NN estimator: $\hat{f}(x) = \hat{\Theta}^T \Phi(x)$
- Estimation error:

$$NN(x) - f(x) = \underbrace{(\hat{\Theta} - \Theta)}_{\Delta\Theta}^T \Phi(x) - \varepsilon_f(x) = \Delta\Theta^T \Phi(x) - \varepsilon_f(x)$$

n^{th} Order Systems with Matched Uncertainties, (continued)

- Stable Reference Model: $\dot{x}_m = A_m x_m + B_m r, \quad (A_m \text{ is Hurwitz})$
- **Control Goal**
 - bounded tracking: $\lim_{t \rightarrow \infty} \|x(t) - x_m(t)\| \leq \varepsilon_x$
- MRAC Design Process
 - choose N and vector of widths $\vec{\sigma}$
 - can be performed off-line in order to incorporate any prior knowledge about the uncertainty
 - design MRAC and evaluate closed-loop system performance
 - repeat previous two steps, if required

n^{th} Order Systems with Matched Uncertainties, (continued)

- Control Feedback: $u = \hat{K}_x^T x + \hat{K}_r^T r + \hat{\Theta}^T \Phi(x)$
 - $(m n + m^2 + m N)$ - parameters to estimate: $\hat{K}_x$, $\hat{K}_r$, $\hat{\Theta}$
- Closed-Loop: $\dot{x} = \left(A + B \Lambda \hat{K}_x^T \right) x + B \Lambda \left(\hat{K}_r^T r + \Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right)$
- Desired Dynamics: $\dot{x}_m = A_m x_m + B_m r$
- Model Matching Conditions
 - there exist ideal gains (K_x, K_r) such that: $\Rightarrow$

$$\begin{aligned} A + B \Lambda K_x^T &= A_m \\ B \Lambda K_r^T &= B_m \end{aligned}$$
 - Note: knowledge of the ideal gains is not required $\Downarrow$

$$\begin{aligned} A + B \Lambda \hat{K}_x^T - A_m &= A + B \Lambda \hat{K}_x^T - A - B \Lambda K_x^T = B \Lambda \left(\hat{K}_x - K_x \right)^T = B \Lambda \Delta K_x^T \\ B \Lambda \hat{K}_r^T - B_m &= B \Lambda \hat{K}_r^T - B \Lambda K_r^T = B \Lambda \left(\hat{K}_r - K_r \right)^T = B \Lambda \Delta \hat{K}_r^T \end{aligned}$$

n^{th} Order Systems with Matched Uncertainties, (continued)

- Tracking Error: $e(t) = x(t) - x_m(t)$
- Error Dynamics:

$$\begin{aligned}
 \dot{e}(t) &= \dot{x}(t) - \dot{x}_m(t) = \\
 &\left(A + B \Lambda \hat{K}_x^T \right) x + B \Lambda \left(\hat{K}_r^T r + \Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right) - A_m x_m - B_m r \pm A_m x \\
 &= A_m (x - x_m) + \left(A + B \Lambda \hat{K}_x^T - A_m \right) x + B \Lambda \left(\hat{K}_r - K_r \right)^T r + B \Lambda \left(\Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right) \\
 &= A_m e + B \Lambda \left(\Delta K_x^T x + \Delta K_r^T r + \Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right)
 \end{aligned}$$

- Remarks
 - estimation error $\varepsilon_f(x)$ is bounded, as long as $x \in X$
 - need to keep x within X

n^{th} Order Systems with Matched Uncertainties, (continued)

- Lyapunov Function Candidate

$$V(e, \Delta K_x, \Delta K_r, \Delta \Theta) = e^T P e + \text{trace}(\Delta K_x^T \Gamma_x^{-1} \Delta K_x |\Lambda|) + \text{trace}(\Delta K_r^T \Gamma_r^{-1} \Delta K_r |\Lambda|) + \text{trace}(\Delta \Theta^T \Gamma_{\Theta}^{-1} \Delta \Theta |\Lambda|)$$

- where: $\text{trace}(S) \triangleq \sum s_{ii}$
- $|\Lambda| \triangleq \text{diag}(|\lambda_1| \quad \dots \quad |\lambda_m^i|)$ is diagonal matrix with positive elements
- $\Gamma_x = \Gamma_x^T > 0$, $\Gamma_r = \Gamma_r^T > 0$, $\Gamma_{\Theta} = \Gamma_{\Theta}^T > 0$ are symmetric positive definite matrices
- $P = P^T > 0$ is unique symmetric positive definite solution of the algebraic Lyapunov equation $P A_m + A_m^T P = -Q$
 - $Q = Q^T > 0$ is any symmetric positive definite matrix

n^{th} Order Systems with Matched Uncertainties, (continued)

- Time-derivative of the Lyapunov function

$$\begin{aligned}
 \dot{V} &= \dot{e}^T P e + e^T P \dot{e} \\
 &+ 2 \operatorname{trace} \left(\Delta K_x^T \Gamma_x^{-1} \dot{\hat{K}}_x |\Lambda| \right) + 2 \operatorname{trace} \left(\Delta K_r^T \Gamma_r^{-1} \dot{\hat{K}}_r |\Lambda| \right) + 2 \operatorname{trace} \left(\Delta \Theta^T \Gamma_{\Theta}^{-1} \dot{\hat{\Theta}} |\Lambda| \right) \\
 &= \left(A_m e + B \Lambda \left(\Delta K_x^T x + \Delta K_r^T r + \Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right) \right)^T P e \\
 &+ e^T P \left(A_m e + B \Lambda \left(\Delta K_x^T x + \Delta K_r^T r + \Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right) \right) \\
 &+ 2 \operatorname{trace} \left(\Delta K_x^T \Gamma_x^{-1} \dot{\hat{K}}_x |\Lambda| \right) + 2 \operatorname{trace} \left(\Delta K_r^T \Gamma_r^{-1} \dot{\hat{K}}_r |\Lambda| \right) + 2 \operatorname{trace} \left(\Delta \Theta^T \Gamma_{\Theta}^{-1} \dot{\hat{\Theta}} |\Lambda| \right) \\
 &= e^T (A_m P + P A_m) e \\
 &+ 2 e^T P B \Lambda \left(\Delta K_x^T x + \Delta K_r^T r + \Delta \Theta^T \Phi(x) - \varepsilon_f(x) \right) \\
 &+ 2 \operatorname{trace} \left(\Delta K_x^T \Gamma_x^{-1} \dot{\hat{K}}_x |\Lambda| \right) + 2 \operatorname{trace} \left(\Delta K_r^T \Gamma_r^{-1} \dot{\hat{K}}_r |\Lambda| \right) + 2 \operatorname{trace} \left(\Delta \Theta^T \Gamma_{\Theta}^{-1} \dot{\hat{\Theta}} |\Lambda| \right)
 \end{aligned}$$

n^{th} Order Systems with Matched Uncertainties, (continued)

- Time-derivative of the Lyapunov function

$$\begin{aligned}\dot{V} = & -e^T Q e - 2e^T P B \Lambda \varepsilon_f(x) \\ & + 2e^T P B \Lambda \Delta K_x^T x + 2 \text{trace} \left(\Delta K_x^T \Gamma_x^{-1} \dot{\hat{K}}_x |\Lambda| \right) \\ & + 2e^T P B \Lambda \Delta K_r^T r + 2 \text{trace} \left(\Delta K_r^T \Gamma_r^{-1} \dot{\hat{K}}_r |\Lambda| \right) \\ & + 2e^T P B \Lambda \Delta \Theta^T \Phi(x) + 2 \text{trace} \left(\Delta \Theta^T \Gamma_{\Theta}^{-1} \dot{\hat{\Theta}} |\Lambda| \right)\end{aligned}$$

- Using trace identity: $a^T b = \text{trace}(b a^T)$

- Example: $\underbrace{e^T P B \Lambda}_{a^T} \underbrace{\Delta K_x^T x}_b = \text{trace} \left(\underbrace{\Delta K_x^T x}_b \underbrace{e^T P B \Lambda}_{a^T} \right)$

n^{th} Order Systems with Matched Uncertainties, (continued)

- Time-derivative of the Lyapunov function

$$\begin{aligned}\dot{V} = & -e^T Q e - 2e^T P B \Lambda \varepsilon_f(x) \\ & + 2 \text{trace} \left(\Delta K_x^T \left\{ \Gamma_x^{-1} \dot{\hat{K}}_x + x e^T P B \text{sgn}(\Lambda) \right\} |\Lambda| \right) \\ & + 2 \text{trace} \left(\Delta K_r^T \left\{ \Gamma_r^{-1} \dot{\hat{K}}_r + r e^T P B \text{sgn}(\Lambda) \right\} |\Lambda| \right) \\ & + 2 \text{trace} \left(\Delta \Theta^T \left\{ \Gamma_{\Theta}^{-1} \dot{\hat{\Theta}} + \Phi(x) e^T P B \text{sgn}(\Lambda) \right\} |\Lambda| \right)\end{aligned}$$

- Problem

- choose adaptive parameters $\hat{K}_x, \hat{K}_r, \hat{\Theta}$ such that time-derivative $\dot{V}$ becomes negative definite outside of a compact set in the error state space, and all parameters remain bounded for all future times

n^{th} Order Systems with Matched Uncertainties, (continued)

- Suppose that we choose adaptive laws:

$$\begin{aligned}\dot{\hat{K}}_x &= -\Gamma_x x e^T P B \operatorname{sgn}(\Lambda) \\ \dot{\hat{K}}_r &= -\Gamma_r r e^T P B \operatorname{sgn}(\Lambda) \\ \dot{\hat{\Theta}} &= -\Gamma_{\Theta} \Phi(x) e^T P B \operatorname{sgn}(\Lambda)\end{aligned}$$

- Then we get:

$$\dot{V} = -e^T Q e - 2e^T P B \Lambda \varepsilon_f(x) \leq -\lambda_{\min}(Q) \|e\|^2 + 2\|e\| \|P B\| \lambda_{\max}(\Lambda) \varepsilon$$

- Consequently, $\dot{V} < 0$ outside of the set

$$E \triangleq \left\{ e : \|e\| \leq \frac{2\|P B\| \lambda_{\max}(\Lambda) \varepsilon}{\lambda_{\min}(Q)} \right\}$$

- Unfortunately, inside E parameter errors may grow out of bounds, (for $e \in E$, $\dot{V}$ IS NOT necessarily negative!)

How to Keep Adaptive Parameters Bounded?

- σ - modification:

$$\begin{aligned}\dot{\hat{K}}_x &= -\Gamma_x \left(x e^T P B + \sigma_x \hat{K}_x \right) \text{sgn}(\Lambda) \\ \dot{\hat{K}}_r &= -\Gamma_r \left(r e^T P B + \sigma_r \hat{K}_r \right) \text{sgn}(\Lambda) \\ \dot{\hat{\Theta}} &= -\Gamma_{\Theta} \left(\Phi(x) e^T P B + \sigma_{\Theta} \hat{\Theta} \right) \text{sgn}(\Lambda)\end{aligned}$$

- e - modification:

$$\begin{aligned}\dot{\hat{K}}_x &= -\Gamma_x \left(x e^T P B + \sigma_x \|e^T P B\| \hat{K}_x \right) \text{sgn}(\Lambda) \\ \dot{\hat{K}}_r &= -\Gamma_r \left(r e^T P B + \sigma_r \|e^T P B\| \hat{K}_r \right) \text{sgn}(\Lambda) \\ \dot{\hat{\Theta}} &= -\Gamma_{\Theta} \left(\Phi(x) e^T P B + \sigma_{\Theta} \|e^T P B\| \hat{\Theta} \right) \text{sgn}(\Lambda)\end{aligned}$$

- Modifications add damping to adaptive laws
 - damping controlled by choosing $\sigma_x, \sigma_r, \sigma_{\Theta} > 0$
 - there is a trade off between adaptation rate and damping

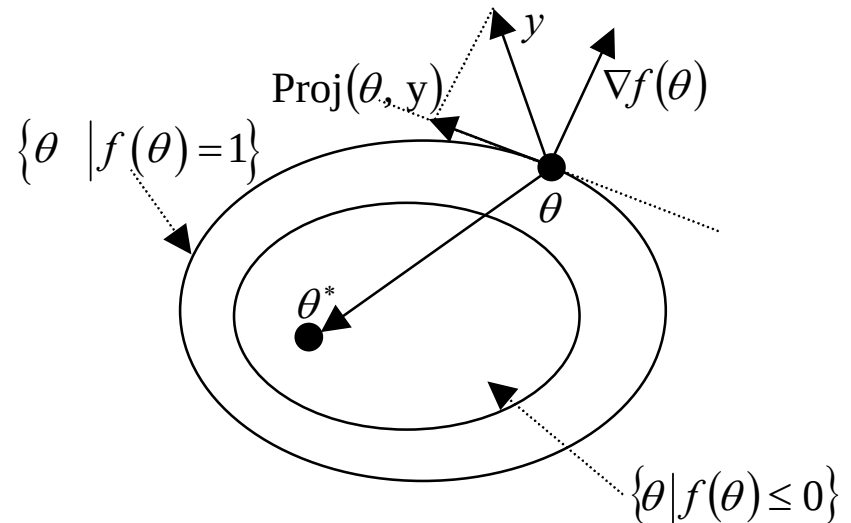
Introducing Projection Operator

- Requires no damping terms
- Designed to keep NN weights within pre-specified bounds
- Maintains negative values of the Lyapunov function time-derivative outside of subset:
$$E \triangleq \left\{ e : \|e\| \leq \frac{2\|P B\| \lambda_{\max}(\Lambda) \varepsilon}{\lambda_{\min}(Q)} \right\}$$
 - the size of E defines tracking tolerance
 - the size of E can be controlled!

Projection Operator

- Function $f(\theta)$ defines pre-specified parameter domain boundary
- Example:

$$f(\theta) = \frac{\|\theta\|^2 - \theta_{\max}^2}{\varepsilon_{\theta} \theta_{\max}^2}$$



$\{f(\theta) \leq 0\} \Rightarrow \{\|\theta\| \leq \theta_{\max}\} \Rightarrow \theta$ is within bounds

$\{0 < f(\theta) \leq 1\} \Rightarrow \{\|\theta\| \leq \sqrt{1 + \varepsilon_{\theta}} \theta_{\max}\} \Rightarrow \theta$ is within $(\sqrt{1 + \varepsilon_{\theta}})\%$ of bounds

$\{f(\theta) > 1\} \Rightarrow \{\|\theta\| > \sqrt{1 + \varepsilon_{\theta}} \theta_{\max}\} \Rightarrow \theta$ is outside of bounds

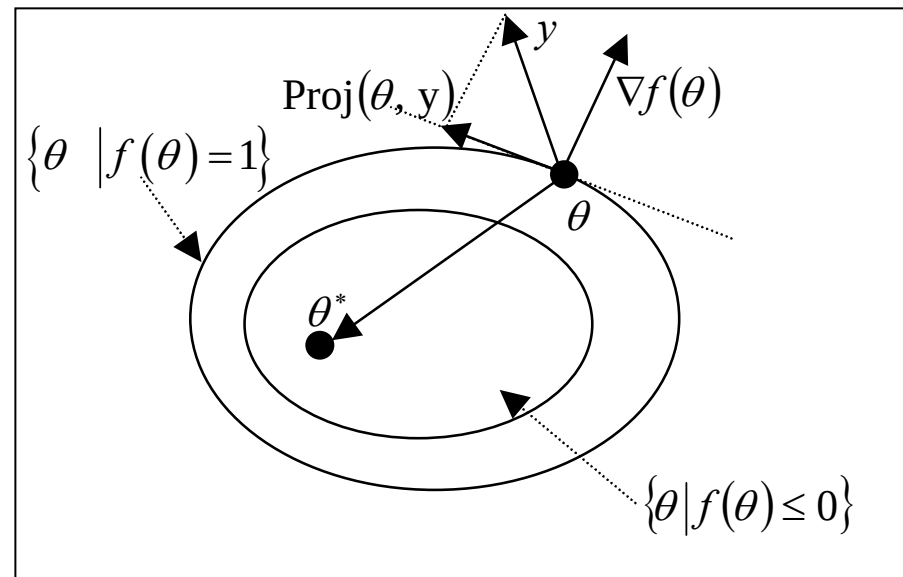
- $\theta_{\max}$ specifies boundary
- ε_{θ} specifies boundary tolerance

Projection Operator, (continued)

- Definition:**

$$\text{Proj}(\theta, y) = \begin{cases} y - \frac{\nabla f(\theta)(\nabla f(\theta))^T}{\|\nabla f(\theta)\|^2} y f(\theta), & \text{if } f(\theta) > 0 \text{ and } y^T \nabla f(\theta) > 0 \\ y, & \text{if not} \end{cases}$$

- Depends on (θ, y)
- Does not alter y if θ is within the pre-specified bounds: $\longrightarrow \|\theta\| \leq \theta_{\max}$
- Gradient: $\longrightarrow \nabla f(\theta) = \frac{2\theta}{\varepsilon_{\theta} \theta_{\max}^2}$
- In $\{0 \leq f(\theta) \leq 1\}$ the operator subtracts gradient vector $\nabla f(\theta)$ (normal to the boundary) from y
 - get a *smooth* transition from y for $\lambda = 0$ to a tangent vector field for $\lambda = 1$



- Important Property**

$$(\theta - \theta^*)^T (\text{Proj}(\theta, y) - y) \leq 0$$

Lyapunov Function Time-Derivative with Projection Operator

- Make trace terms semi-negative AND keep parameters bounded:

$$\begin{aligned}
 \dot{V} = & -e^T Q e - 2e^T P B \Lambda \varepsilon_f(x) \\
 & + 2 \operatorname{trace} \left(\Delta K_x^T \left\{ \underbrace{\Gamma_x^{-1} \dot{\hat{K}}_x}_{\operatorname{Proj}(\hat{K}_x, y)} + \underbrace{x e^T P B \operatorname{sgn}(\Lambda)}_{-y} \right\} |\Lambda| \right) \\
 & + 2 \operatorname{trace} \left(\Delta K_r^T \left\{ \underbrace{\Gamma_r^{-1} \dot{\hat{K}}_r}_{\operatorname{Proj}(\hat{K}_r, y)} + \underbrace{r e^T P B \operatorname{sgn}(\Lambda)}_{-y} \right\} |\Lambda| \right) \\
 & + 2 \operatorname{trace} \left(\Delta \Theta^T \left\{ \underbrace{\Gamma_\Theta^{-1} \dot{\hat{\Theta}}}_{\operatorname{Proj}(\hat{\Theta}, y)} + \underbrace{\Phi(x) e^T P B \operatorname{sgn}(\Lambda)}_{-y} \right\} |\Lambda| \right)
 \end{aligned}$$

Adaptation with Projection

- Modified adaptive laws:

$$\begin{aligned}\dot{\hat{K}}_x &= \Gamma_x \text{Proj}\left(\hat{K}_x, -x e^T P B \text{sgn}(\Lambda)\right) \\ \dot{\hat{K}}_r &= \Gamma_r \text{Proj}\left(\hat{K}_r, -r e^T P B \text{sgn}(\Lambda)\right) \\ \dot{\hat{\Theta}} &= \Gamma_{\Theta} \text{Proj}\left(\hat{\Theta}, -\Phi(x) e^T P B \text{sgn}(\Lambda)\right)\end{aligned}$$

- Projection Operator, its bounds and tolerances are defined column-wise
- Lyapunov function time-derivative:

$$\dot{V} \leq -e^T Q e - 2e^T P B \Lambda \varepsilon_f(x) \leq -\lambda_{\min}(Q) \|e\|^2 + 2\|e\| \|P B\| \lambda_{\max}(\Lambda) \varepsilon$$

- Adaptive parameters stay within the pre-specified bounds, while $\dot{V} < 0$

outside of the set:

$$E \triangleq \left\{ e : \|e\| \leq \frac{2\|P B\| \lambda_{\max}(\Lambda) \varepsilon}{\lambda_{\min}(Q)} \right\}$$

Example: Projection Operator, (scalar case)

- Scalar adaptive gain: $\dot{\hat{k}} = \gamma \text{Proj}(\hat{k}, -x e \text{sgn}(b))$
- Pre-specified parameter domain boundary:

– using function:

$$f(\hat{k}) = \frac{\hat{k}^2 - k_{\max}^2}{\varepsilon k_{\max}^2} \longrightarrow \nabla f(\hat{k}) = f'(\hat{k}) = \frac{2\hat{k}}{\varepsilon k_{\max}^2}$$

$$\{f(\hat{k}) \leq 0\} \Rightarrow \{|\hat{k}| \leq k_{\max}\} \Rightarrow \hat{k} \text{ is within bounds}$$

$$\{0 < f(\hat{k}) \leq 1\} \Rightarrow \{|\hat{k}| \leq \sqrt{1+\varepsilon} k_{\max}\} \Rightarrow \hat{k} \text{ is within } (\sqrt{1+\varepsilon})\% \text{ of bounds}$$

$$\{f(\hat{k}) > 1\} \Rightarrow \{|\hat{k}| > \sqrt{1+\varepsilon} k_{\max}\} \Rightarrow \hat{k} \text{ is outside of bounds}$$

– Projection Operator:

$$y = -x e \text{sgn}(b)$$



$$\text{Proj}(\hat{k}, y) = \begin{cases} y(1 - f(\hat{k})), & \text{if } f(\hat{k}) > 0 \text{ and } y f'(\hat{k}) > 0 \\ y, & \text{if not} \end{cases}$$

Example: Projection Operator, (scalar case) (continued)

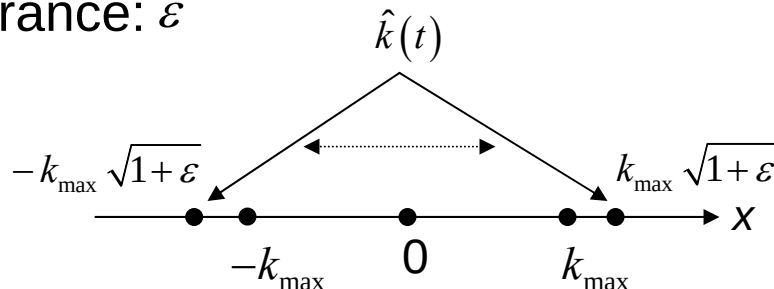
- Adaptive Law, ($b > 0$):

$$\dot{\hat{k}} = \begin{cases} -x e(1 - f(\hat{k})), & \text{if } [f(\hat{k})] > 0 \text{ and } [x e f'(\hat{k})] < 0 \\ -x e, & \text{if not} \end{cases}$$

where: $f(\hat{k}) = \frac{\hat{k}^2 - k_{\max}^2}{\varepsilon k_{\max}^2}$

- Geometric Interpretation

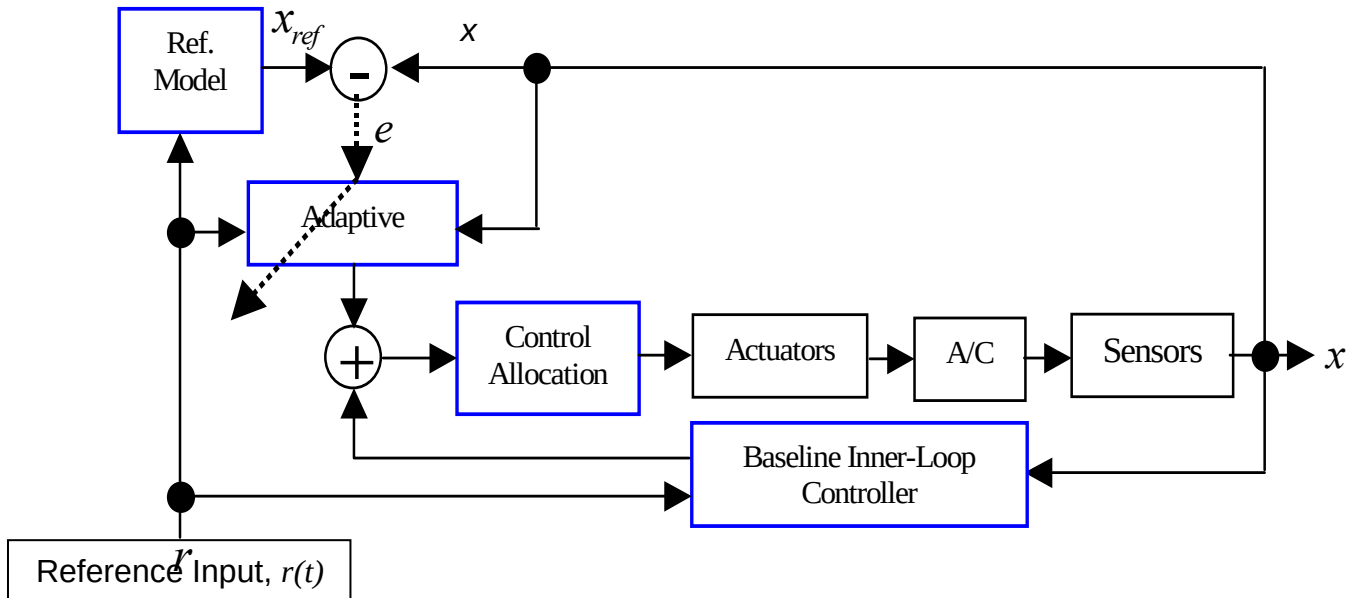
- adaptive parameter $\hat{k}(t)$ changes within the pre-specified interval
- interval bound: $k_{\max}$
- Bound tolerance: ε



Adaptive Augmentation Design

- Nominal Control:
$$u_{nom} = F_x^T x + F_r^T r$$
- Adaptive Control:
$$u = \hat{K}_x^T x + \hat{K}_r^T r + \hat{\Theta}^T \Phi(x)$$
- Augmentation:
$$\begin{aligned} u &= \hat{K}_x^T x + \hat{K}_r^T r + \hat{\Theta}^T \Phi(x) \pm u_{nom} \\ &= u_{nom} + \underbrace{(\hat{K}_x - F_x)^T}_{\hat{D}_x} x + \underbrace{(\hat{K}_r - F_r)^T}_{\hat{D}_r} r + \hat{\Theta}^T \Phi(x) \\ &= u_{nom} + \hat{D}_x^T x + \hat{D}_r^T r + \hat{\Theta}^T \Phi(x) \end{aligned}$$
- Incremental Adaptation:
$$\begin{aligned} \dot{\hat{D}}_x &= \Gamma_x \text{Proj}(\hat{D}_x, -x e^T P B \text{sgn}(\Lambda)), \quad \hat{D}_x = 0_{n \times m} \\ \dot{\hat{D}}_r &= \Gamma_r \text{Proj}(\hat{D}_r, -r e^T P B \text{sgn}(\Lambda)), \quad \hat{D}_r = 0_{m \times m} \\ \dot{\hat{\Theta}} &= \Gamma_{\Theta} \text{Proj}(\hat{\Theta}, -\Phi(x) e^T P B \text{sgn}(\Lambda)), \quad \hat{\Theta} = 0_{N \times m} \end{aligned}$$

Adaptive Augmentation Block-Diagram



- Reference Model provides desired response
- Nominal Baseline Controller
- Adaptive Augmentation
 - Dead-Zone modification prevents adaptation from changing nominal closed-loop dynamics
 - Projection Operator bounds adaptation parameters / gains

Adaptive Control using Sigmoidal NN

- System Dynamics: $\dot{x} = A x + B \Lambda (u - f(x)), \quad x \in R^n, \quad u \in R^m$
 - $A \in R^{n \times n}$, $\Lambda = \text{diag}(\lambda_1 \dots \lambda_m) \in R^{m \times m}$ are constant unknown matrices
 - $B \in R^{M \times m}$ is known constant matrix, and $M \geq m$
 - $\forall i = 1, \dots, m \quad \text{sgn}(\lambda_i)$ is known

- **Approximation of uncertainty:**

$$f(x) = W^T \vec{\sigma}(V^T \mu) + \varepsilon_f(x), \quad \mu = (x^T \quad 1)^T, \quad \varepsilon_f(x) \in R^m$$

- matrix of constant unknown Inner-Layer weights:

$$V = \begin{bmatrix} \vec{v}_1 & \dots & \vec{v}_N \\ \theta_1 & \dots & \theta_N \end{bmatrix} \in R^{(n+1) \times N}$$

- matrix of constant unknown Outer-Layer weights:

$$W = \begin{bmatrix} \vec{w}_1 & \dots & \vec{w}_m \\ c_1 & \dots & c_m \end{bmatrix} \in R^{(N+1) \times m}$$

- vector of N sigmoids and a unity:

$$\vec{\sigma}(V^T \mu) = \left(\sigma(\vec{v}_1^T x + \theta_1) \quad \dots \quad \sigma(\vec{v}_N^T x + \theta_N) \quad 1 \right)^T, \quad \text{where: } \sigma(s) = \frac{1}{1 + e^{-s}}$$

Adaptive Control using Sigmoidal NN

- Control Feedback:
$$u = \hat{K}_x^T x + \hat{K}_r^T r + \hat{W}^T \vec{\sigma}(\hat{V}^T \mu)$$
 - $(m n + m^2 + (n + 1) N + (N + 1) m)$ - parameters to estimate: $\hat{K}_x$, $\hat{K}_r$, $\hat{W}$, $\hat{V}$
- Adaptation with Projection, $(\Lambda > 0)$:

$$\begin{cases} \dot{\hat{K}}_x = \Gamma_x \text{Proj}(\hat{K}_x, -x e^T P B) \\ \dot{\hat{K}}_u = \Gamma_u \text{Proj}(\hat{K}_u, -r e^T P B) \\ \dot{\hat{W}} = \Gamma_w \text{Proj}(\hat{W}, (\vec{\sigma}(\hat{V}^T \mu) - \vec{\sigma}'(\hat{V}^T \mu) \hat{V}^T \mu) e^T P B) \\ \dot{\hat{V}} = \Gamma_v \text{Proj}(\hat{V}, \mu e^T P B \hat{W}^T \vec{\sigma}'(\hat{V}^T \mu)) \end{cases}$$
- Provides bounded tracking